

1. Database

What is a database?

A database is a structured place where application data is stored permanently and managed efficiently.

It is a organised collection of data managed by a software called DBMS.

Example

```
CREATE DATABASE employee_management;
```

Now we have a database called:

employee_management

Inside it, we can create tables:

employee_management

```
|  
├── employee  
├── department  
└── designation
```

A database provides **persistent storage**.

If your Python program stops:

```
employees = []
```

the data disappears.

But if you store it in PostgreSQL:

PostgreSQL

↓
Data remains even after application restart

2. Schema

A schema is a logical namespace/container inside a database that organizes database objects such as tables, views, functions, etc.

It is basically the overall description about the database it stores information about the database in dbms.

Easy difference

Database = main container

Schema = logical organization inside database

Table = stores actual records

A Postgres schema is a logical container that organizes database objects such as tables, views, indexes, functions and sequences within a database. It helps group related objects and manage them efficiently.

- Stores related database objects in a single namespace.
- Improves database organization and maintenance.
- Enables better control over user access and permissions.

A PostgreSQL database can contain multiple schemas. A schema is a namespace inside the database used to organize tables and other database objects.

Syntax

Create a Schema

```
CREATE SCHEMA schema_name;
```

we can do operations on schema directly through pgadmin.

3. Table

A table stores data in rows and columns.

Example:

employee

employee_id	employee_name	email	salary
1	Rohit	rohit@gmail.com	60000
2	Raj	raj@gmail.com	50000
3	Priti	priti@gmail.com	70000

The table is:

employee

Its columns are:

employee_id
employee_name
email
salary

```
CREATE TABLE employee (  
    employee_id SERIAL PRIMARY KEY,  
    employee_name VARCHAR(100),  
    email VARCHAR(150),  
    salary DECIMAL(10,2)  
);
```

4. Row

A row represents one complete record in a table.

Example:

employee_id	employee_name	email	salary
1	Rohit	rohit@gmail.com	60000

This entire line is **one row** in emp id 1.

5. Column

A column represents one attribute/property of the records.

Example:

employee_id	employee_name	email	salary
1	Rohit	rohit@gmail.com	60000

The columns are employee_id , employee_name, email, salary.

6. Primary Key

A primary key uniquely identifies each row in a table.

Example:

employee_id is a primary key here

employee_id	employee_name
1	Rohit
2	Rahul
3	Priya

We define:

employee_id SERIAL PRIMARY KEY

Every employee has a unique ID.

Primary key rules

A primary key:

- must be unique
- cannot be NULL
- identifies one record
- normally exists once per table

SQL Example:

```
CREATE TABLE employee (  
    employee_id SERIAL PRIMARY KEY,  
    employee_name VARCHAR(100)  
);
```

7. Foreign Key

A foreign key creates a relationship between tables.

It usually refers to the Primary Key of another table.

It is basically a way to connect our two tables by setting up a relationship between them.

Suppose we have:

Department

department_id	department_name
1	Software Development
2	Data Engineering
3	HR

Employee

employee_id	employee_name	department_id
1	Rohit	2
2	Rahul	2
3	Priya	3

Here:

Employee.department_id

references:

Department.department_id

SQL:

```
FOREIGN KEY (department_id)  
REFERENCES department(department_id)
```

A foreign key helps PostgreSQL maintain **referential integrity**.

8. Constraints

A constraint is a rule that PostgreSQL enforces on your data.

constraints are rules enforced on data columns and tables to prevent invalid data from being entered into the database

Important constraints:

PRIMARY KEY
FOREIGN KEY
NOT NULL
UNIQUE
CHECK

Default

And many more constraints.

NOT NULL

Means a value is required.

```
employee_name VARCHAR(100) NOT NULL
```

This is invalid:

```
INSERT INTO employee(employee_name)  
VALUES (NULL);
```

UNIQUE

Prevents duplicate values.

```
email VARCHAR(150) UNIQUE
```

CHECK

Ensures a condition.

```
age INT CHECK (age >= 18)
```

other:

```
salary DECIMAL(10,2) CHECK (salary > 0)
```

9. Relationships

Relationships describe **how tables are connected to each other**.

It defines the relationships between two tables.

The main types are:

1. One-to-One
 2. One-to-Many
 3. Many-to-Many
-

One-to-One

One record relates to one record.

Person ——— Passport
1 1

One person has one passport.

One-to-Many

One record relates to many records.

This is extremely common.

Our example:

Department
|
1
|
N
↓
Employee

One department can have many employees.

Many-to-Many

Many records relate to many records.

Example:

Student ↔ Course

A student can take many courses.

A course can have many students.

10. Unique Constraints

A UNIQUE constraint prevents duplicate values.

Example:

```
email VARCHAR(150) UNIQUE
```

Suppose:

employee_id	email
1	rohit@gmail.com
2	rahul@gmail.com

Trying:

```
INSERT INTO employee(email)
VALUES ('rohit@gmail.com');
```

will fail because that email already exists.

UNIQUE vs PRIMARY KEY

Primary Key

Used to identify a record.

```
employee_id PRIMARY KEY
```

UNIQUE

Used to ensure a value doesn't repeat.

```
email UNIQUE
```

Example:

```
employee_id → identity
email       → uniqueness
```

11. NULL

NULL means no value is currently available/assigned/known.

It is **not** the same as:

0

and not the same as:

''

Employee example

Rohit doesn't report to another manager.

So:

employee_id	employee_name	manager_id
1	Rohit	NULL
2	Rahul	1
3	Priya	1

For Rohit:

```
manager_id = NULL
```

means:

No manager is assigned to Rohit.

12. Index

A PostgreSQL index is a data structure that helps PostgreSQL find rows faster without scanning the entire table.

Think of it like the **index of a book**.

Without an index:

Table → Check row 1 → row 2 → row 3 → ... → row 1,000,000

With an index:

Index → Quickly find matching rows → Table

Example

Suppose:

```
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100)  
);
```

If you frequently search by email:

```
SELECT * FROM students  
WHERE email = 'rohit@gmail.com';
```

Create an index:

```
CREATE INDEX idx_students_email  
ON students(email);
```

Now PostgreSQL can find the matching row **more efficiently**.

Important

Index

- └─ Makes SELECT/search faster
- └─ Uses additional storage
- └─ Can make INSERT/UPDATE/DELETE slightly slower

Because PostgreSQL also has to update the index when the data changes.

PostgreSQL typically uses a B-tree index by default.

13. Normalization

Normalization is the process of organizing database data to **reduce duplication and maintain data consistency**.

- It is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data in different places), which otherwise unnecessarily increases the size of the database.
- Data redundancy leads to inconsistency problems during insert, delete, and update operations, making data management difficult.
- Normalization involves splitting a table into multiple tables, which must be linked each time a query requires data from the split tables.

Example — Bad Design

Employee

ID	Name	Department	Department_Location
1	Rohit	Data Engineering	Noida
2	Rahul	Data Engineering	Noida

Data Engineering and Noida are repeated.

Normalized Design

Department

department_id	department_name	location
1	Data Engineering	Noida

Employee

employee_id	name	department_id
1	Rohit	1
2	Rahul	1

Now department information is stored only once and connected using `department_id`.

Benefits

- Reduces duplicate data
- Prevents inconsistent data
- Makes updates easier
- Avoids insert/update/delete anomalies
- Creates cleaner table relationships

1NF — First Normal Form

Rule: Each column must contain **atomic (single) values**, and there should be no repeating groups.

Bad:

Employee	Skills
Rohit	Python, SQL, Spark

Good:

Employee	Skill
----------	-------

Rohit		Python
Rohit		SQL
Rohit		Spark

Remember: 1NF = one value per cell

2NF — Second Normal Form

Rule: Must be in 1NF, and every non-key column must depend on the **whole primary key**.

Mainly matters when using a **composite primary key**.

Example:

employee_id | project_id | employee_name | project_name

Primary key:

(employee_id, project_id)

But:

employee_name → depends only on employee_id
project_name → depends only on project_id

So they should be separated:

Employee
employee_id | employee_name

Project
project_id | project_name

Employee_Project
employee_id | project_id

Remember: 2NF = no partial dependency

3NF — Third Normal Form

Rule: Must be in 2NF, and non-key columns should not depend on another non-key column.

Bad:

employee_id | department_id | department_name

Here:

employee_id → department_id → department_name

department_name depends on department_id, not directly on the employee.

Separate:

Employee
employee_id | department_id

Department
department_id | department_name

Remember: 3NF = no transitive dependency

4NF — Fourth Normal Form

Rule: Must be in 3NF and should not contain **independent multi-valued relationships** in the same table.

Bad:

Employee	Skill	Language
Rohit	Python	English
Rohit	Python	Hindi
Rohit	SQL	English
Rohit	SQL	Hindi

Skills and languages are independent of each other, causing unnecessary combinations.

Separate:

Employee_Skill	
Employee	Skill
Rohit	Python
Rohit	SQL

Employee_Language	
Employee	Language
Rohit	English
Rohit	Hindi

Remember: 4NF = no independent multi-valued dependencies

5NF — Fifth Normal Form

Rule: Must be in 4NF and should eliminate redundancy caused by complex **join dependencies**.

It deals with situations where information can be correctly split into multiple tables and reconstructed using joins.

Remember: 5NF = eliminate join dependency/redundancy

14. PostgreSQL Transactions

In **PostgreSQL**, a transaction is a group of SQL operations executed as **one unit of work**.

Basic syntax

BEGIN;

-- SQL operations

COMMIT;

If an error occurs:

ROLLBACK;

Example

Suppose we transfer ₹10,000:

```
BEGIN;
```

```
UPDATE account  
SET balance = balance - 10000  
WHERE account_id = 1;
```

```
UPDATE account  
SET balance = balance + 10000  
WHERE account_id = 2;
```

```
COMMIT;
```

If something fails before **COMMIT**:

```
ROLLBACK;
```

PostgreSQL undoes the changes made in that transaction.

Important PostgreSQL commands

BEGIN	→ Start transaction
COMMIT	→ Save all changes
ROLLBACK	→ Undo all changes
SAVEPOINT	→ Create a point to partially rollback

Example:

```
BEGIN;
```

```
UPDATE employee  
SET salary = salary + 5000  
WHERE employee_id = 1;
```

```
SAVEPOINT salary_update;
```

```
UPDATE employee  
SET salary = salary + 5000  
WHERE employee_id = 2;
```

```
ROLLBACK TO salary_update;
```

```
COMMIT;
```

Here, the first update remains, while the second update is undone.

ACID Properties

Property	Meaning
Atomicity	All operations succeed or all are undone
Consistency	Database remains valid
Isolation	Concurrent transactions don't improperly interfere
Durability	Committed changes persist

Explanation of Relationships

1. Department → Employee — 1:M

One department can have **many employees**, but each employee belongs to one department.

Example:

Data Engineering → Rohit, Rahul, Priti

```
department.department_id
      ↓
employee.department_id
```

2. Designation → Employee — 1:M

One designation can be assigned to **many employees**, but each employee has one designation.

Example:

Data Engineer → Rohit, Rahul

```
designation.designation_id
      ↓
employee.designation_id
```

3. Manager → Employee — 1:M

One manager can manage **many employees**, but each employee can have one manager.

Example:

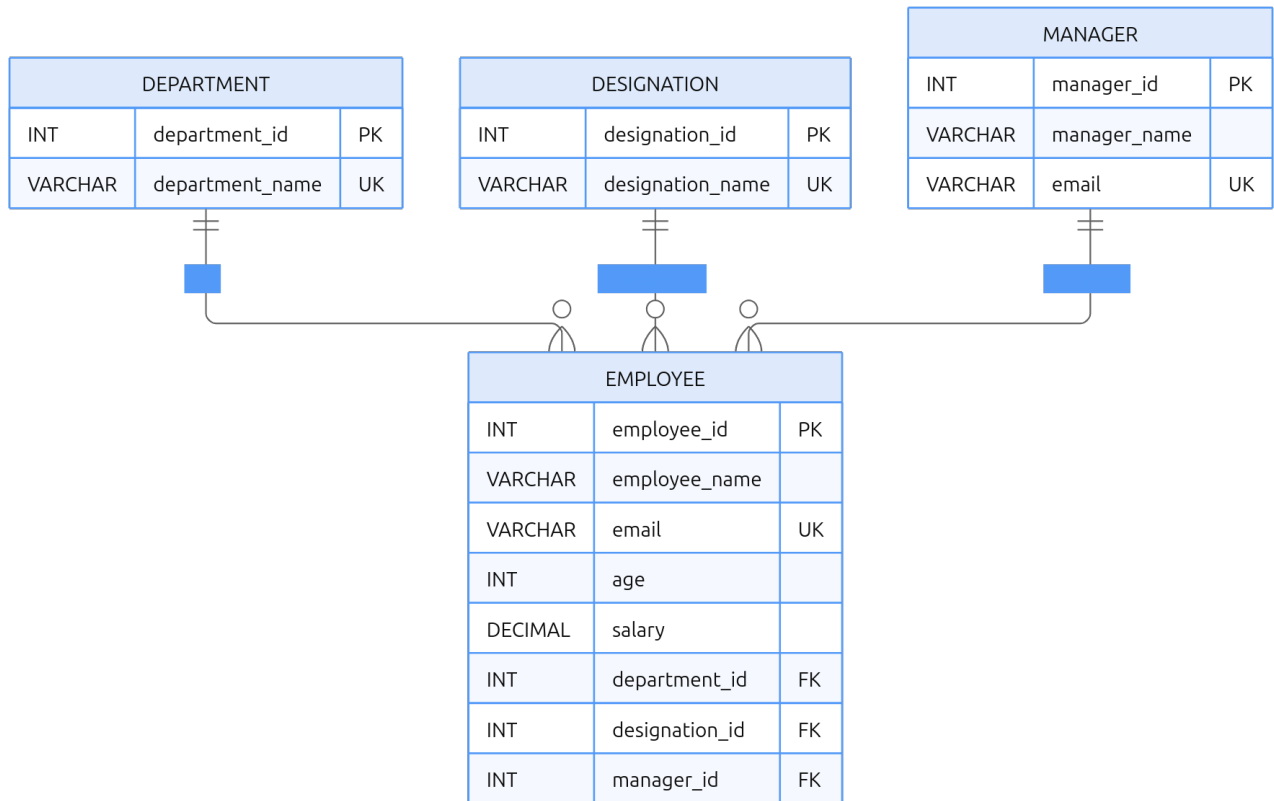
Rohit → Rahul, Priya

```
manager.manager_id
      ↓
employee.manager_id
```

Summary

Department	→ Employee	= 1:M
Designation	→ Employee	= 1:M
Manager	→ Employee	= 1:M

These relationships are implemented using **foreign keys** in the employee table.



Sample data

1. Department — 10 Records

```
INSERT INTO department (department_name)
VALUES
('Software Development'),
('Data Engineering'),
('Human Resources'),
('Finance'),
('Marketing'),
('Sales'),
('Operations'),
('Information Technology'),
('Cyber Security'),
('Quality Assurance');
```

2. Designation — 10 Records

```
INSERT INTO designation (designation_name)
VALUES
('Software Engineer'),
('Data Engineer'),
```

```
('HR Manager'),
('Finance Manager'),
('Marketing Executive'),
('Sales Executive'),
('Operations Manager'),
('DevOps Engineer'),
('Security Engineer'),
('QA Engineer');
```

3. Manager — 10 Records

```
INSERT INTO manager (manager_name, email)
VALUES
('Amit Sharma', 'amit@company.com'),
('Priya Singh', 'priya@company.com'),
('Rahul Verma', 'rahul@company.com'),
('Neeraj Kumar', 'neeraj@company.com'),
('Anjali Gupta', 'anjali@company.com'),
('Vivek Sharma', 'vivek@company.com'),
('Pooja Verma', 'pooja@company.com'),
('Rakesh Singh', 'rakesh@company.com'),
('Karan Mehta', 'karan@company.com'),
('Sneha Kapoor', 'sneha@company.com');
```

4. Employee — 10 Records

```
INSERT INTO employee
(employee_name, email, age, salary, department_id, designation_id, manager_id)
VALUES
('Rohit Singh', 'rohit@company.com', 22, 60000, 2, 2, 1),
('Neha Gupta', 'neha@company.com', 24, 55000, 1, 1, 1),
('Arjun Kumar', 'arjun@company.com', 25, 58000, 2, 2, 1),
('Kavya Sharma', 'kavya@company.com', 28, 65000, 3, 3, 2),
('Vikas Verma', 'vikas@company.com', 30, 70000, 4, 4, 3),
('Ankit Yadav', 'ankit@company.com', 23, 50000, 5, 5, 4),
('Simran Kaur', 'simran@company.com', 26, 52000, 6, 6, 5),
('Aditya Mehta', 'aditya@company.com', 27, 68000, 8, 8, 6),
('Nisha Patel', 'nisha@company.com', 29, 72000, 9, 9, 7),
('Manish Gupta', 'manish@company.com', 31, 62000, 10, 10, 8);
```

Verify

```
SELECT * FROM department;
```

```
SELECT * FROM designation;
```

```
SELECT * FROM manager;
```

```
SELECT * FROM employee;
```